

Validate IPv4 Address

Java Code + Visualization

1. Java Code

```
class Solution {
    public boolean isValid(String s) {

        String[] parts = s.split("\\.", -1);

        if (parts.length != 4) {
            return false;
        }

        for (int i = 0; i < parts.length; i++) {

            if (parts[i].isEmpty()) {
                return false;
            }

            if (parts[i].length() > 1 &&
                parts[i].charAt(0) == '0') {
                return false;
            }

            int num = 0;

            for (int j = 0; j < parts[i].length(); j++) {

                if (parts[i].charAt(j) < '0' ||
                    parts[i].charAt(j) > '9') {
                    return false;
                }

                num = num * 10 +
                    (parts[i].charAt(j) - '0');
            }

            if (num > 255) {
                return false;
            }
        }

        return true;
    }
}
```

2. Visualization

Valid IPv4: 192.168.1.1

192.168.1.1
↓
192 | 168 | 1 | 1
↓ ↓ ↓ ↓
P1 P2 P3 P4

Exactly 4 parts → ■

3. Check Empty Parts

192..1.1
↓
192 | "" | 1 | 1

↑

4. Check Leading Zero

```
Valid:    192.168.0.1
Invalid:  192.168.01.1
```

01
↑
Leading zero → ■

0 → ■
01 → ■
001 → ■

5. Check Digits

19a

```
1 → digit ■
9 → digit ■
a → digit ■
```

6. Build the Number

For 192:

```
num = 0
```

$$\begin{aligned} 1 &\rightarrow 0 \times 10 + 1 = 1 \\ 9 &\rightarrow 1 \times 10 + 9 = 19 \\ 2 &\rightarrow 19 \times 10 + 2 = 192 \end{aligned}$$
$$192 \leq 255 \rightarrow \blacksquare$$

7. Check Range

[illegible]

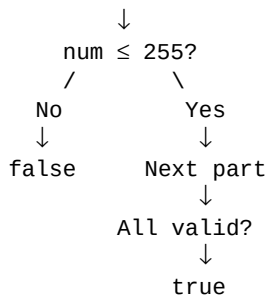
192 → ■
255 → ■
256 → ■
300 → ■

8. Complete Flow

```

graph TD
    A[IPv4 String] --> B[Split using "."]
    B --> C[Exactly 4 parts?]
    C -- No --> D[false]
    C -- Yes --> E[Check each part]
    E --> F[Empty part?]
    F --> G[Leading zero?]
    G --> H[Check digits]
    H --> I[Build the number]

```



9. Examples

Input	Visualization	Output
192.168.1.1	192 168 1 1	true
192.168.01.1	01 → leading zero	false
192.168.1.256	256 > 255	false
192..1.1	Empty part	false

10. Complexity

Time → $O(n)$

Space → $O(n)$

n = length of the IP string.

Space includes the array created by `split()`.